

Sahrane Guassemi ESGI M2 IW 2025-2066

GO -> Google -> 2009

Langage impopulaire

Langage compilé $\neq$ pas d'interpréteur entre l'instruction et le processeur

Langage compilé :

Point +

Performant

Point -

Complexité

Rigidité ?

re-compilé

fuite de mémoire

Risque accru pour les amateurs

Un nouveau langage disruptif serait potentiellement un langage qui a pour objectif

- Bas niveau
- Web
- Mobile

IA -> Python -> tensor flow -> réseau de neurones

Langage go, objet ? l'héritage a été mis en place pour éviter de refaire la même chose

L'héritage est ce qu'hérite un enfant de son parent, la composition permet de composer un ensemble

Dans le langage Go, l'un des principal changement c'est la **composition** en POO

Un des principaux avantages du go est la simplicité

Rapidité au niveau des performances et de la compilation

Le go va imposer pas mal de contrainte comme des syntaxes imposé a tous, par exemple si une accolade est mal placé le code ne pas pas compiler, là ou en C, même si c'est

Ici on a une rigidité qui permet une simplification (import fmt)

En Go il n'y a plus de try catch

Le multi threading en go est porté par le Go Routines (dead lock)

Go a une approche pragmatique (l'inverse de créatif) et impose une seule façon idiomatique de faire

Go pousse à utiliser des best practices

L'avantage de tout ça c'est que Go apporte une stabilité et de la compatibilité dans le temps

Go a été conçu pour être le plus efficace possible, compromis entre la performance, la simplicité et la rapidité de développement et de compilation

IL y a 25 mots syntaxiques dans Go

Go va être utilisé pour les micro service et les API REST

2) Outils CLI

3) Infrastructure cloud

4) Demon system

```
birthDate.go
1  package main
2
3  import (
4      "fmt"
5      "time"
6  )
7
8  func main() {
9      // je met ma date de naissance
10     birthDate := time.Date(1997, time.April, 16, 0, 0, 0, 0, time.Local)
11     // je récupère la date actuelle
12     now := time.Now()
13     // je récupère l'année de la date actuelle et je vais soustraire l'année de ma date e naissance
14     age := now.Year() - birthDate.Year()
15     fmt.Println("J'ai", age, "ans")
16 }
```

```
sahrane@UbuSahrane:~/ESGI/Master 2/GoLang$ go run birthDate.go
J'ai 29 ans
sahrane@UbuSahrane:~/ESGI/Master 2/GoLang$
```

go build -ldflags "-s -w" -> expliquer les options de build, et comparé avec -ldflags "-s -w"

Go build compile les package en un fichier binaire exécutable comme on peut le voir dans la capture d'écran ci-dessous :

```
sahrane@UbuSahrane:~/ESGI/Master 2/GoLang$ ls
birthDate  birthDate.go  hello.go
sahrane@UbuSahrane:~/ESGI/Master 2/GoLang$ ./birthDate
J'ai 29 ans
```

l'utilisation des options -ldflags "-s -w" est une transmission au linker et non pas au compilateur le -s permet d'omettre la table des symbol et le -w la table des infos DWARF qui est la table des infos de debug

les deux cumulé permettre d'optimiser le fichier binaire, par exemple pour un environnement de production c'est mieux adapté

Les types en GO :

ENTIERS

Int 8 | uint8

int 16 | uint 16

int 32 | uint 32

| uint 64

Les flottants

float32

float64

Booleans

Var bool1 = true

String

Var text = " Cours ESGI"

-> Concat

-> Add

Les strings sont indexables

Les spéciaux :

Alias

Byte = uint 8

Rune = int 32

La valeur "null" habituel s'appelle "Nil"

La déclaration de variable est principalement "var"

iota ->

iota est un identifiant go spécial qui est utilisable qu'avec le bloc const, il permet de générer une suite automatiser de valeur entiere

Par principe, iota commence par 0 et incrémente de 1 à chaque fois

Elle est utile les énumérations ou les flags ou rapport d'erreurs ordonnées

Faut savoir aussi que la rpésence d'un iota réinitialise a 0 donc dans une suite elle permet de réordonner par exemple une suite et de la réajuste avec la présence du bloc iota

le format fmt contraint toujours d'avoir le langage indenté de la même façon, cela permet d'éviter des conversions implicites

Les entiers ont des valeurs par défauts (0 == 0 pour de vrai) , pas de undefined pas de nul, Go garanti la valeur à 0.

les tableaux slice sont des valeurs modifiables

le := est une déclaration fonctionnel mais UNIQUEMENT dans les fonctions

Les développeur Go sont appelés Goffeurs

L'approche de GO est l'erreur comme valeur, on voit visuellement dans le code qu'une opération peut échouer

le fallthrough n'est pas automatique en Go comme ça l'est pour C, Java, Javascript, faut faire attention car on pourrait attendre un comportement

pour le langage Go, dans la POO on est dans la composition (exemple des LEGO)

La visibilité dans le langage Go dépend uniquement de la première lettre d'un identifiant. On ne retrouve donc pas les mots-clés habituels comme "public", "private"

Une fonction avec une majuscule est exportable/public et accessible depuis n'importe quel package

Une fonction en minuscule ne sera pas exportable et donc privée et sera uniquement utilisée dans le package courant

Go se base sur le camelCase -> exemple "maFonction" ne sera pas exportable mais "MaFonction" le sera

Cette règle s'applique à tous les identifiants (types, fonctions, variables, champs de structures

Par contre même si une structure est exportée, ses champs en minuscule eux restent inaccessibles de l'extérieur

En Go, le mot-clé "defer" permet de systématiquement exécuter du code dans une fonction avant le "return" et cela même si le cas d'une erreur remontée dans une exécution en amont.

Dans le cas de plusieurs defer dans une fonction, c'est le dernier qui sera le premier à être exécuté